

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
CURSO: *Manejo e Implementación de Archivos*



MANUAL DE USUARIO

Analizador Léxico y Sintáctico de Comandos EXT2

Nombre:

ADILZON ALFREDO VELÁSQUEZ HERNÁNDEZ

Carné No.:

201908076

Auxiliar: Williams Bramlley Constanza Oscar

Guatemala, 11 de Agosto 2026

Índice

1. Introducción
2. Requisitos previos
3. Estructura del proyecto
4. Instalación y puesta en marcha
 - 4.1 Puesta en marcha del backend (API REST en C++)
 - 4.2 Puesta en marcha del frontend (interfaz en React)
5. Uso de la interfaz
 - 5.1 Pantalla principal
 - 5.2 Cómo escribir un comando
 - 5.3 Cómo interpretar los resultados
6. Ejemplos de uso
7. Resolución de problemas comunes
8. Recomendaciones finales

1. Introducción

El presente manual tiene como propósito guiar al usuario en la instalación, puesta en marcha y uso correcto del sistema “Analizador Léxico y Sintáctico de Comandos EXT2”, desarrollado como parte de la Práctica 1 del curso Manejo e Implementación de Archivos, Segundo Semestre 2026.

El sistema permite ingresar comandos relacionados con la administración de discos y sistemas de archivos EXT2 (MKDISK, RMDISK, FDISK, MOUNT, MKFS, MKUSR, RMUSR y MKFILE), analizando únicamente su estructura léxica y sintáctica. El programa no ejecuta ninguna acción real sobre archivos o particiones; su función es identificar errores de escritura, parámetros inválidos y problemas de gramática en los comandos ingresados.

La solución está compuesta por un backend desarrollado en C++, encargado del análisis de los comandos, y un frontend desarrollado en React, que permite al usuario interactuar con el sistema de forma sencilla e intuitiva. La comunicación entre ambos componentes se realiza mediante una API REST.

2. Requisitos previos

Antes de poner en marcha el sistema, es necesario contar con lo siguiente instalado en el equipo:

- Sistema operativo Linux, instalado de forma física (no se admite el uso de máquina virtual).
- Compilador g++ con soporte para el estándar C++17.
- Herramienta make.
- Node.js y npm (versión 18 o superior recomendada) para ejecutar el frontend en React.
- Un navegador web moderno (Google Chrome, Mozilla Firefox, entre otros).
- Puerto 8080 disponible para el backend y puerto 5173 disponible para el frontend (puertos por defecto).

3. Estructura del proyecto

El proyecto se organiza de forma modular, separando las declaraciones (encabezados) de las implementaciones, y separando el backend del frontend:

```
practica/
├── include/      (implementación: scanner.cpp, validador.cpp, api.cpp)
├── lib/          (encabezados: scanner.h, validador.h, api.h, httpplib.h,
json.hpp)
├── src/          (main.cpp, main_server.cpp, ejecutable, servidor)
├── frontend/     (proyecto React: App.jsx, App.css, package.json)
└── Makefile
```

El programa “ejecutable” corresponde a la versión de consola del analizador, mientras que “servidor” corresponde al servidor de la API REST que utiliza la interfaz web.

4. Instalación y puesta en marcha

El sistema requiere dos componentes corriendo de manera simultánea: el backend (servidor de la API REST en C++) y el frontend (interfaz web en React). A continuación, se detalla el procedimiento para levantar ambos.

4.1 Puesta en marcha del backend (API REST en C++)

1. Abrir una terminal y ubicarse dentro de la carpeta src/ del proyecto.
2. Ejecutar el siguiente comando para eliminar compilaciones previas (recomendado la primera vez):

```
make clean
```

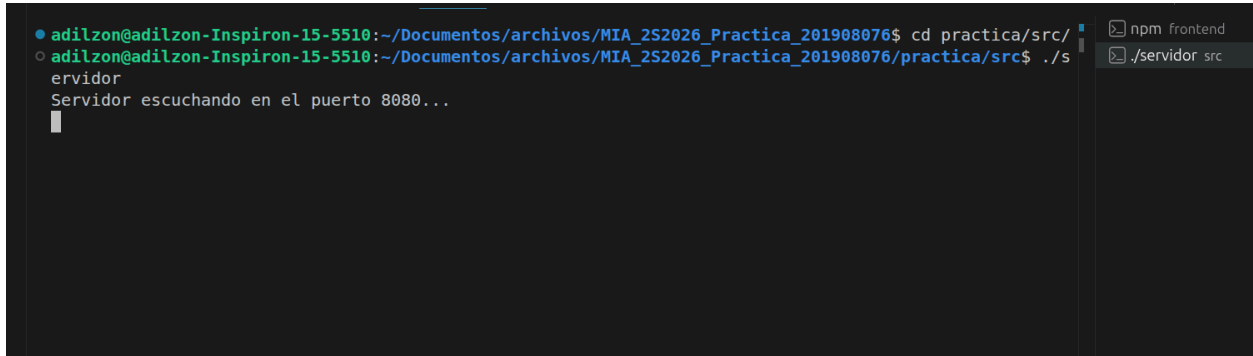
3. Compilar el proyecto:

```
make
```

- Este paso genera dos programas: “ejecutable” (versión de consola) y “servidor” (API REST).
Iniciar el servidor con:

```
./servidor
```

- Si todo funciona correctamente, la terminal mostrará el mensaje “Servidor escuchando en el puerto 8080...”. Esta terminal debe permanecer abierta mientras se utilice el sistema; cerrarla detiene el backend.



```

● adilzon@adilzon-Inspiron-15-5510:~/Documentos/archivos/MIA_252026_Practica_201908076$ cd practica/src/
○ adilzon@adilzon-Inspiron-15-5510:~/Documentos/archivos/MIA_252026_Practica_201908076/practica/src$ ./s
  servidor
  Servidor escuchando en el puerto 8080...
  
```

4.2 Puesta en marcha del frontend (interfaz en React)

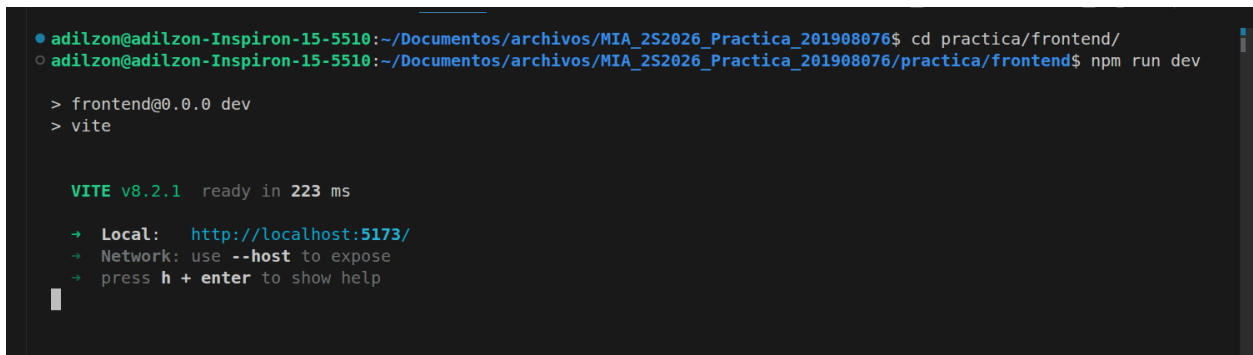
- Abrir una segunda terminal, sin cerrar la del backend, y ubicarse dentro de la carpeta frontend/.
- Instalar las dependencias del proyecto (únicamente es necesario la primera vez):

```
npm install
```

- Iniciar el servidor de desarrollo de React:

```
npm run dev
```

- Copiar la dirección que se muestra en la terminal (por defecto `http://localhost:5173`) y abrirla en el navegador web.



```

● adilzon@adilzon-Inspiron-15-5510:~/Documentos/archivos/MIA_252026_Practica_201908076$ cd practica/frontend/
○ adilzon@adilzon-Inspiron-15-5510:~/Documentos/archivos/MIA_252026_Practica_201908076/practica/frontend$ npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.2.1 ready in 223 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
  
```

5. Uso de la interfaz

5.1 Pantalla principal

Al abrir la aplicación en el navegador, se muestra la pantalla principal del analizador, compuesta por tres elementos principales: un área de texto para ingresar los comandos, un botón “Analizar” y una consola de resultados donde se muestra el resultado del análisis de cada comando.

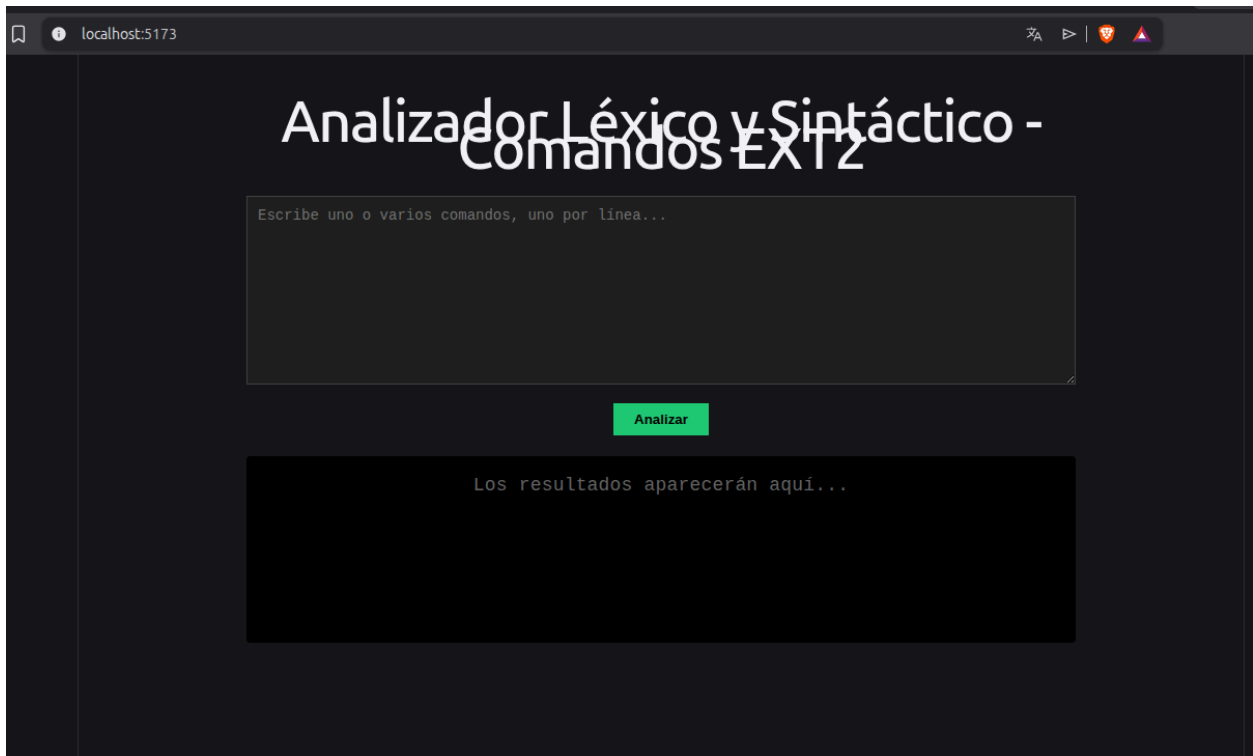


Figura 1. Pantalla principal del sistema, mostrando el área de entrada de comandos, el botón “Analizar” y la consola de resultados vacía.

5.2 Cómo escribir un comando

Para analizar uno o varios comandos, el usuario debe seguir las siguientes indicaciones:

- Escribir un comando por línea dentro del área de texto.
- Cada comando debe iniciar con el nombre del comando, seguido de sus parámetros, cada uno con el formato `-parametro=valor`.
- Si un valor contiene espacios en blanco (por ejemplo, una ruta), debe encerrarse entre comillas dobles.
- Las líneas que inician con el símbolo `#` se consideran comentarios y son ignoradas por el analizador.
- Una vez escritos los comandos, se debe presionar el botón “Analizar” para procesar todas las líneas ingresadas.

Ejemplo de comandos ingresados en el área de texto:

```
mkdisk -size=3000 -unit=K -path=/home/user/Disco1.mia  
fdisk -size=300 -path=/home/Disco1.mia -name=Particion1  
mount -path=/home/Disco1.mia -name=Particion1
```

5.3 Cómo interpretar los resultados

Después de presionar “Analizar”, la consola de resultados muestra, para cada línea ingresada, si el comando es sintácticamente válido o si contiene errores:

- Un comando marcado en verde con el símbolo ✓ indica que la sintaxis del comando es correcta.
- Un comando marcado en rojo con el símbolo ✗ indica que se encontraron errores; debajo del comando se listan todos los errores detectados (parámetros faltantes, valores inválidos o parámetros desconocidos).

```

$ mkdisk -size=20 -path="/home/archivos/Discos/mis
archivos/Disco5.dk" -unit=M -fit=WF
Comando válido

$ mkdisk -param=x -size=30 -
path=/home/archivos/archivos/fase1/Disco.dk
Parametro desconocido: "-param"

```

Figura 2. Consola de resultados mostrando un comando válido (en verde) y un comando con errores detectados (en rojo).

6. Ejemplos de uso

A continuación, se presentan ejemplos de comandos y el resultado que el sistema debe mostrar al analizarlos:

Comando ingresado	Resultado esperado
<code>mkdisk -size=3000 -unit=K -path=/home/user/Disco1.mia</code>	Válido
<code>mkdisk -unit=K -path=/home/user/Disco1.mia</code>	Error: falta -size
<code>mkdisk -size=3000 -fit=XX -path=/home/user/Disco1.mia</code>	Error: -fit inválido
<code>rmdisk -path=/home/mis discos/Disco4.mia</code>	Válido
<code>fdisk -size=300 -path=/home/Disco1.mia -name=Particion1</code>	Válido
<code>mount -path=/home/Disco2.mia -name=Part2</code>	Válido
<code>mkfs -type=fast -id=341A</code>	Error: -type inválido
<code>mkusr -user=user2 -grp=admin</code>	Error: falta -pass
<code>rmusr -user=user1</code>	Válido
<code>mkfile -size=15 -path=/home/user/docs/a.txt -r</code>	Válido
<code>comandoinvalido</code>	Error: comando no reconocido

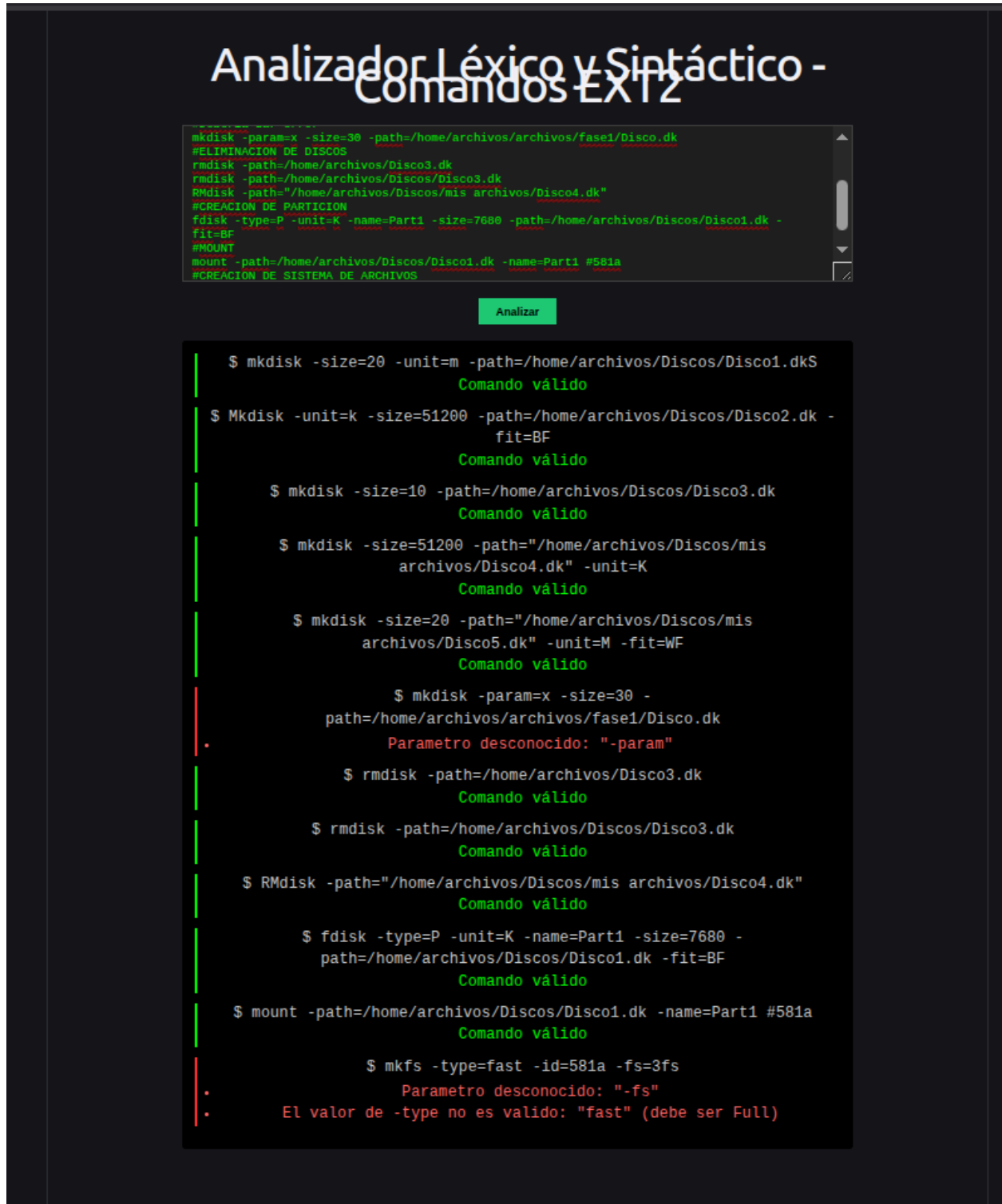


Figura 3. Resultado en la interfaz al analizar un lote de comandos, combinando casos válidos y con errores.

7. Resolución de problemas comunes

En esta sección se listan los problemas más frecuentes que puede encontrar el usuario, junto con su solución:

Problema	Causa probable	Solución
El mensaje “No se pudo conectar con el servidor” aparece en los resultados.	El backend (./servidor) no está en ejecución, o se cerró la terminal donde corría.	Verificar que el backend esté corriendo. Abrir una terminal, ubicarse en src/ y ejecutar ./servidor nuevamente.
La página del navegador aparece en blanco o no carga.	El frontend no fue iniciado o faltan sus dependencias.	En la carpeta frontend/, ejecutar npm install y luego npm run dev, y volver a abrir la dirección indicada.
En la consola del navegador aparece un error relacionado con CORS.	El backend no está corriendo en la dirección/puerto que espera el frontend.	Confirmar que ./servidor esté escuchando en el puerto 8080 y que la constante API_URL en App.jsx apunte a http://localhost:8080/analizar.
Se muestra “COMANDO INVALIDO” en un comando que parece correcto.	Error de escritura en el nombre del comando o uso de un comando no contemplado.	Verificar la ortografía. Los únicos comandos reconocidos son: MKDISK, RMDISK, FDISK, MOUNT, MKFS, MKUSR, RMUSR y MKFILE.
Una ruta o valor con espacios genera un error inesperado.	El valor con espacios en blanco no fue encerrado entre comillas dobles.	Encerrar el valor completo entre comillas dobles, por ejemplo: -path="/home/mis discos/Disco1.mia".
Aparece el error “Parametro desconocido”.	Se escribió un parámetro que no corresponde al comando, o se usó una abreviatura no soportada (por ejemplo -u en lugar de -unit).	Revisar la tabla de parámetros del comando en el enunciado y utilizar el nombre completo del parámetro.
Al ejecutar make aparece un error de compilación relacionado con httpLib.h o json.hpp.	Las librerías externas no se descargaron dentro de la carpeta lib/.	Descargar nuevamente httpLib.h y json.hpp dentro de lib/ y ejecutar make clean seguido de make.
El backend indica que el puerto 8080 ya está en uso.	Existe otra instancia del servidor ejecutándose previamente.	Cerrar el proceso anterior (Ctrl + C en la terminal donde corría) antes de iniciar ./servidor nuevamente.

8. Recomendaciones finales

- Mantener abiertas las dos terminales (backend y frontend) mientras se utilice el sistema.
- Guardar los comandos de prueba en archivos de texto para poder repetir las pruebas fácilmente.
- Revisar siempre la consola de resultados completa, ya que un mismo comando puede tener más de un error.
- Ante cualquier duda adicional sobre el funcionamiento interno del sistema, consultar el Manual Técnico.